

一、排序先看三件事

读排序代码时，先问三个问题：

- 1. 排序方向：升序还是降序。
- 2. 比较依据：直接比较元素，还是比较某个字段、某个表达式。
- 3. 每轮目标：本轮确定最大值、最小值，还是把一个新元素插入已有有序区。

排序不会改变数据元素本身，只会改变元素之间的先后次序。

- 数组排序：通常通过交换或移动元素，进行物理重排。
- 链表排序：通常不搬动节点的数据域，而是修改 next 指针。

常见排序对照：

排序	每轮核心动作	容易考的点
选择排序	从未排序区选出最小/最大值	min_i / max_i 更新条件
冒泡排序	相邻元素不断交换	内层循环范围、j+1 越界
插入排序	把当前元素插入有序区	数组元素后移、链表指针修改
计数排序	统计每个值出现次数	计数数组下标、还原顺序
桶排序	按范围分桶，再桶内整理	桶编号计算、桶内顺序
归并排序	先分后合并	合并两个有序段
快速排序	选基准，把小大分开	基准、左右指针、递归范围

二、选择排序

选择排序的思路：第 i 轮在 i ~ n-1 中找最小值，把它放到位置 i。

选择排序：第 1 轮中 min_i 如何移动

① 初始化

5

2

4

1

3

i / min_i

i=0, 先假设 a[0] 最小

② 比较 j=1

5

2

4

1

3

i min_i

2 < 5, min_i 从 0 移到 1

③ 比较 j=2

5

2

4

1

3

i min_i

4 < 2 不成立, min_i 不动

④ 比较 j=3、4

5

2

4

1

3

i min_i

找到 1, 最终 min_i=3

⑤ 交换

1

2

4

5

3

i

交换 a[i] 与 a[min_i], 位置 0 已确定

图中黄色表示本轮位置 `i`，红色表示当前找到的最小值位置 `min_i`。扫描结束后只交换一次，因此选择排序的交换次数通常少于冒泡排序。

1. 标准升序模板

```
a = [5, 2, 4, 1, 3]
n = len(a)
for i in range(n - 1):
    min_i = i
    for j in range(i + 1, n):
        if a[j] < a[min_i]:
            min_i = ①
    a[i], a[min_i] = a[min_i], a[i]
print(a)
```

追踪表:

轮次 <code>i</code>	未排序区	找到的最小值	交换后数组
0	5, 2, 4, 1, 3		
1			
2			
3			

2. 选择排序变式：按距离排序

如果要求按“离目标值 24 的距离从小到大”排序。

```
a = [30, 18, 25, 23, 21, 28]
target = 24
for i in range(len(a) - 1):
    k = i
    for j in range(i + 1, len(a)):
        if _____:
            k = j
    a[i], a[k] = a[k], a[i]
```

三、冒泡排序

冒泡排序的思路：通过相邻比较交换，让最大值或最小值一步步“冒”到边界。

先用三步快速判断，不要背四段代码

看到冒泡排序代码，固定按“方向、升降序、索引范围”三步判断。

第一步：看 `range`，判断从哪边往哪边

```
for j in range(0, n - 1 - i):
```

`j` 逐渐增大，是从前往后比较；通常比较 `a[j]` 和 `a[j+1]`。

```
for j in range(n - 1, i, -1):
```

第三个参数是 `-1`，`j` 逐渐减小，是从后往前比较；通常比较 `a[j]` 和 `a[j-1]`。

快速判断：

- `range` 中没有 `-1`，`j` 递增：从前往后。
- `range` 的第三个参数是 `-1`，`j` 递减：从后往前。

第二步：先找真正的左值 `L` 和右值 `R`，再判断升降序

if `a[j] > a[j-1]` :交换

交换后的结果 `a[j] < a[j-1]` 前一个数大，降序

第三步：range 被挖空时，直接用未排序区的左右边界填

考试通常把 range 挖空，例如：

```
for j in range(____①____):  
    if a[j] > a[j + 1]:  
        ...
```

$0 \leq j \leq j+1 \leq \text{len}-1$

j 取交集即可

或者：

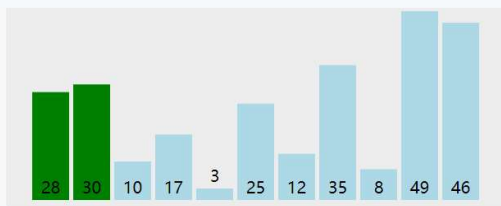
```
for j in range(____①____, ____②____, ____③____):  
    if a[j] > a[j - 1]:  
        ...
```

1. 从前往后冒泡：每轮把最大值放到右边

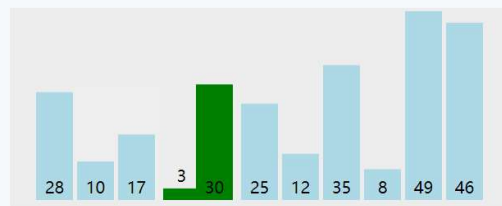
```
a = [5, 2, 4, 1, 3]  
n = len(a)  
for i in range(n - 1):  
    for j in range(0, n - 1 - i):  
        if a[j] > a[j + 1]:  
            a[j], a[j + 1] = a[j + 1], a[j]
```

冒泡排序：截取一轮中的四个关键时刻

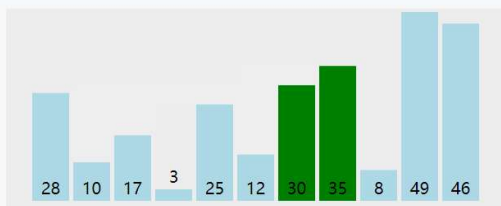
① 从左端开始



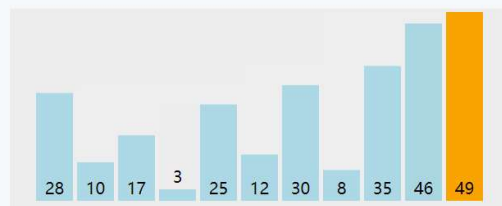
② 较大值继续右移



③ 接近本轮右边界



④ 本轮最大值已确定



绿色表示当前正在比较的一对相邻元素，橙色表示本轮已经确定的最大值。

2. 从后往前冒泡的降序：每轮把最大值放到左边

```
a = [5, 2, 4, 1, 3]  
n = len(a)  
for i in range(n - 1):  
    for j in range(____, ____, ____):  
        if a[j] > a[j - 1]:  
            a[j], a[j - 1] = a[j - 1], a[j]
```

从右往左比较时，若右边的数更大，就与左边的数交换。较大的数会不断向左移动，因此每轮结束后，左侧增加一个已经确定的位置。

轮次 i	j 的范围	本轮确定
0	n-1 ~ 1	最大值到位置 0
1	n-1 ~ 2	次大值到位置 1

3. 从前往后冒泡的降序：每轮把最小值放到右边

```

a = [5, 2, 4, 1, 3]
n = len(a)
for i in range(n - 1):
    for j in range(0, ____①____):
        if a[j] ____②____ a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]

```

从左往右比较时，若左边的数更小，就与右边的数交换。较小的数会不断向右移动，因此每轮结束后，右侧增加一个已经确定的位置。

轮次 i	j 的范围	本轮确定
0	0 ~ n-2	最小值到位置 n-1
1	0 ~ n-3	次小值到位置 n-2

4. 从后往前冒泡的升序：每轮把最小值放到左边

```

a = [5, 2, 4, 1, 3]
n = len(a)
for i in range(n - 1):
    for j in range(____①____, ____②____, -1):
        if a[j] < a[j - 1]:
            a[j], a[j - 1] = ____③____

```

从右往左比较时，若右边的数更小，就与左边的数交换。较小的数会不断向左移动，因此每轮结束后，左侧增加一个已经确定的位置。

轮次 i	j 的范围	本轮确定
0	n-1 ~ 1	最小值到位置 0
1	n-1 ~ 2	次小值到位置 1

四种写法可以统一判断：

遍历方向	代码中的条件	改写为 L、R	排序结果	每轮确定的位置
从前往后	a[j] > a[j+1]	L > R	升序	最大值到右边
从后往前	a[j] > a[j-1]	R > L, 即 L < R	降序	最大值到左边
从前往后	a[j] < a[j+1]	L < R	降序	最小值到右边
从后往前	a[j] < a[j-1]	R < L, 即 L > R	升序	最小值到左边

5. 冒泡排序的稳定性

如果两个元素关键字相等时不交换，那么冒泡排序是稳定（原数据的先后顺序）的。

```

if key(a[j]) > key(a[j + 1]):
    a[j], a[j + 1] = a[j + 1], a[j]

```

如果把条件写成 $\geq$ ，相等元素也会交换，稳定性会被破坏。

6. 交换次数与逆序对

升序排列时，如果前面的数大于后面的数，这一对数称为一个逆序对。

例如 $[3, 1, 2]$ 中有两对逆序对： $(3, 1)$ 和 $(3, 2)$ 。冒泡排序每交换一次相邻逆序对，逆序对数量就减少 1，因此：

冒泡排序的交换次数 = 原序列的逆序对数量。

填空：

序列 $[4, 1, 3, 2]$ 中共有 个逆序对，所以用标准升序冒泡排序时，共交换 次。

7. 冒泡排序优化

优化一：没有交换就提前结束

```
for i in range(n - 1):
    flag = False
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            flag = ①
    if not flag:
        ②
```

flag 的含义：本轮是否发生过交换。若一整轮没有交换，说明数组已经有序，不必继续。

优化二：缩小到“最后一次交换位置”

如果一轮中最后一次交换发生在位置 last，那么 last 右侧已经有序，下一轮不必再比较。

```
end = n - 1
while end > 0:
    last = 0
    for j in range(end):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
            last = ①
    end = ②
```

flag 优化判断“是否可以直接结束”；last 优化进一步记录“下一轮比较到哪里为止”。

8. 冒泡循环范围判断题

以下代码哪一个不能保证升序？

```
# A
for i in range(n - 1):
    for j in range(n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]

# B
for i in range(n - 1):
    for j in range(n - 1, i, -1):
        if a[j] < a[j - 1]:
            a[j], a[j - 1] = a[j - 1], a[j]

# C
for i in range(n - 1):
    for j in range(i + 1, n):
        if a[i] > a[j]:
            a[i], a[j] = a[j], a[i]

# D
for i in range(n - 1):
    for j in range(i, n - 1 - i):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
```

判断方法：看每一轮是否覆盖了必须比较的相邻位置，尤其注意 $j+1$ 是否越界、左右边界是否过早收缩。

9. 局部排序范围

考试中常故意让内层循环不从 0 开始，或不遍历到 $n-1$ 。判断实际排序范围时，不要只看 j ，还要看参与比较的是 $a[j]$ 与 $a[j+1]$ ，还是 $a[j]$ 与 $a[j-1]$ 。

例一：从前往后，使用 $j+1$

```
for j in range(2, n - i - 1):
    if a[j] > a[j + 1]:
        a[j], a[j + 1] = a[j + 1], a[j]
```

- j 的范围：2 ~ $n-i-2$ 。
- 还访问了 $j+1$ ，所以实际数据下标范围：2 ~ $n-i-1$ 。

例二：从后往前，使用 $j-1$

```
for j in range(n - 2, 3, -1):
    if a[j] < a[j - 1]:
        a[j], a[j - 1] = a[j - 1], a[j]
```

- j 的范围： $n-2$ ~ 4。
- 还访问了 $j-1$ ，所以实际数据下标范围：3 ~ $n-2$ 。

综合判断

```
for i in range(1, n - 1):
    for j in range(1, n - i - 1):
        if a[j] > a[j - 1]:
            a[j], a[j - 1] = a[j - 1], a[j]
```

填写：

- j 递增，所以比较方向是从 _____ 往 _____。
- 代码中左值 L 是 _____，右值 R 是 _____。
- 交换条件改写为 L _____ R ，所以结果为 _____ 序。
- 第一轮中 j 的范围是 _____。
- 第一轮实际参与的数据下标范围是 _____。

最后自检：把第一轮中的 j 值依次写出来，并列出每次访问的两个下标。如果手算结果与上面的范围不一致，以逐次列出的下标为准。

四、插入排序

插入排序的思路：左边维护一个有序区，每次把右边第一个元素插入到有序区中正确位置。

1. 数组插入排序

数组插入排序会移动元素。

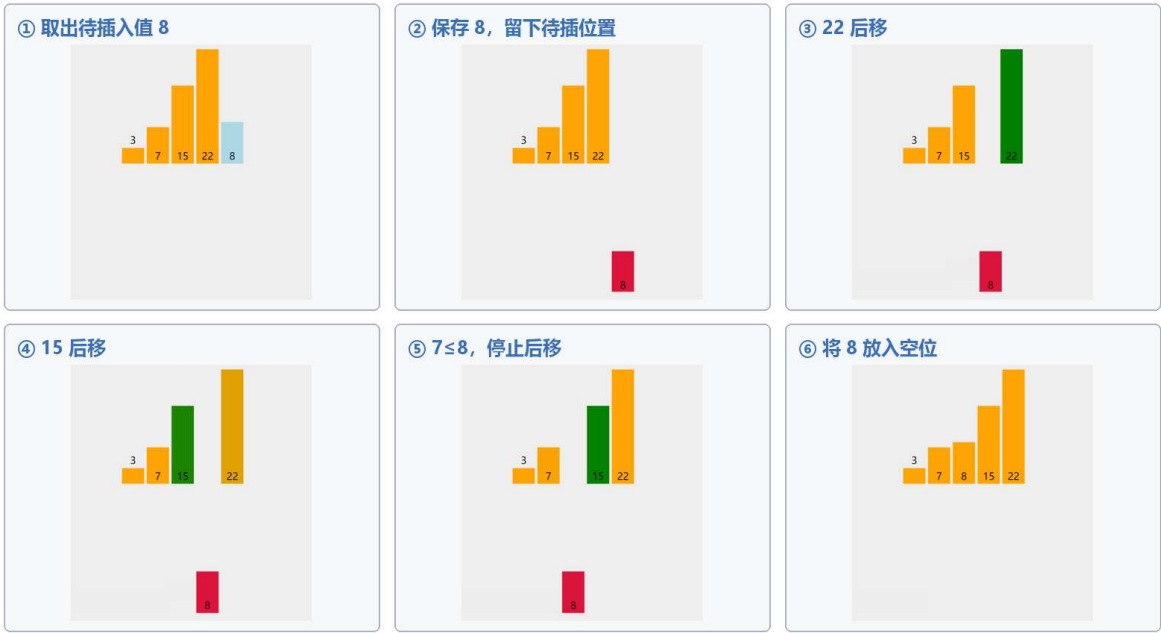
```
a = [4, 2, 3, 1, 5]
for i in range(1, len(a)):
    t = a[i]
    j = i - 1
    while j >= 0 and a[j] > t:
        a[j + 1] = _____①_____
        j -= 1
    a[j + 1] = _____②_____
```

以插入 3 为例：

步骤	有序区	t	j 指向	动作
开始	2, 4	3	4 的位置	
比较	2, 4	3	4 的位置	4 后移
停止	2, _ , 4	3	2 的位置	3 插入空位

下面把“插入 8”这一轮拆成六个关键步骤。打印时可以直接观察：先保存待插入值，再让较大的元素依次后移，最后把待插入值放入空位。

数组插入排序：把 8 插入有序区 [3, 7, 15, 22]



2. 有序数组中单个元素改变

若数组原本已经升序，只把 `a[pos]` 增加 1，那么左侧元素不会比它更大，变化后的元素只可能向__移动。

```

t = a[pos] + 1
i = pos
while i < n - 1 and ____①____:
    a[i] = a[i + 1]
    i += 1
a[i] = ____②____

```

3. 链表插入排序

链表插入排序不会移动数组元素，而是修改 `next` 指针。

链表插入排序：重点观察“插入 3”时 p、q 的变化

原链表把 3 和 5 的位置对调：4 -> 2 -> 3 -> 1 -> 5。节点中只写数值域，箭头表示 next。



场景：前两个节点 4、2 已经插入成有序链表 2 -> 4；现在 p 指向待插入节点 3。

1 让 q 从有序链表表头开始，寻找 3 的插入位置



2 q 停在 2：下一个节点 4 比 3 大，所以 3 应插在 2 和 4 之间



图中重点观察插入 3 时的变量变化：p 始终指向待插入节点，q 在已排序链表中寻找插入位置。找到位置后，要先让 p 接上 q 原来的后继，再让 q 接上 p。

```
data = [[4, 1], [2, 2], [3, 3], [1, 4], [5, -1]]
head = 0
sorted_head = -1
p = head
while p != -1:
    nxt = ____①____
    if sorted_head == -1 or data[p][0] < data[sorted_head][0]:
        data[p][1] = ____②____
        sorted_head = ____③____
    else:
        q = sorted_head
        while data[q][1] != -1 and data[data[q][1]][0] <= data[p][0]:
            q = ____④____
        data[p][1] = ____⑤____
        data[q][1] = ____⑥____
    p = ____⑦____
```

链表插入排序的关键：

1. 先保存原链表后继 nxt，否则一改 data[p][1] 就找不到下一个待处理节点。
2. 若插到表头，修改 sorted_head。
3. 若插到中间，先让新节点接原后继，再让 q 接新节点。

五、桶排序

桶排序先按范围分桶，再在桶内整理，最后按桶顺序收集。

桶排序：先分桶，再进行桶内排序

原数据

87

62

91

73

68

100

按十位分桶

6 号桶

62

68

7 号桶

73

8 号桶

87

9 号桶

91

10 号桶

100

桶内有多个数据时，仍要先排好桶内顺序，再按桶号收集。

收集结果

62

68

73

87

91

100

例：分数范围 0~100，按十位分桶。

```
score = [87, 62, 91, 73, 68, 100]
bucket = [[] for i in range(11)]
for x in score:
    k =         ①
    bucket[k].append(x)
ans = []
for b in bucket:
    b.sort() # <--- 必须加上这一步（也可以用插入排序等）
    for x in b:
        ans.append(x)
```

这里的桶编号 k 代表分数段：0~9 放 0 号桶，10~19 放 1 号桶，以此类推。

桶排序适合数据范围清楚、分布较均匀的情况。若桶内还要排序，就需要在每个桶中再用一种排序方法。

在信息技术考试题中，代码 `bucket[x] += 1` 有时也被称为“桶排序思想”。这种写法的每个桶对应一个具体数值，本质上与下一节的计数排序相同。读题时应以代码为准：

- 桶中保存数据列表：通常还要进行桶内排序。
- 桶中只保存出现次数：按数值下标计数并还原。

六、计数排序

计数排序适合整数范围不大时使用。核心是统计每个值出现了几次。

计数排序：统计、计数、还原

① 遍历原数组

3 1 2 3 0 2

读到 x, 就执行 $\text{cnt}[x] += 1$

下标就是数值，不是原数组位置

② 得到计数数组

1 1 2 2

0 1 2 3

上格：出现次数

下标：对应的数值

③ 按值依次输出

0 1 2 2 3 3

$\text{cnt}[0]=1 \rightarrow$ 输出 1 个 0

$\text{cnt}[2]=2 \rightarrow$ 输出 2 个 2

```
a = [3, 1, 2, 3, 0, 2]
cnt = [0] * 4
for x in a:
    cnt[____①____] += 1

ans = []
for value in range(len(cnt)):
    for k in range(cnt[value]):
        ans.append(____②____)
```

追踪表：

value	cnt[value]	输出几个
0		
1		
2		
3		

如果题目要求稳定计数排序，就不能只按次数输出，还要用前缀和计算每个元素应该放到结果数组的位置。

七、归并排序

归并排序的核心是先把数组分成两半，分别排好，再合并两个有序数组。

1. 合并两个有序数组

```
left = [1, 4, 6]
right = [2, 3, 5]
i = j = 0
ans = []
while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        ans.append(left[i])
        i += 1
    else:
        ans.append(right[j])
        j += 1
while i < len(left):
    ans.append(____①____)
    i += 1
while j < len(right):
    ans.append(____②____)
    j += 1
```

归并排序：合并两个有序段



比较时只看两个有序段当前最前面的元素。较小者进入结果数组，对应指针向右移动；其中一段用完后，直接追加另一段的剩余元素。

2. 归并排序骨架

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)
```

递归题要看出口、拆分、合并三件事。归并排序的真正排序发生在“合并两个有序段”时。

八、快速排序

快速排序的核心是选一个基准，把比基准小的放左边，比基准大的放右边，再递归处理左右两边。

1. 简化写法

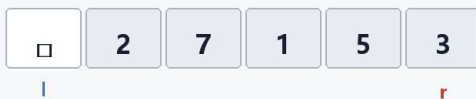
```
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[0]
    left = []
    right = []
    for i in range(1, len(a)):
        if a[i] < pivot:
            left.append(____①____)
        else:
            right.append(____②____)
    return quick_sort(left) + [pivot] + quick_sort(right)
```

2. 原地分区的读法

很多考试代码不会创建 left/right, 而是用左右指针交换。

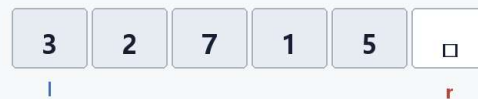
快速排序：挖坑法完成第一轮分区

① 取出基准



pivot=4, 左端形成空位

② r 向左找小值



找到 3, 填入左边空位; 空位移到右边

③ l 向右找大值



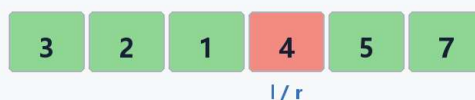
找到 7, 填入右边空位; 空位移到位置 2

④ r 再向左找小值



找到 1, 填入左边空位; l、r 即将相遇

⑤ 放回基准



pivot 放到相遇位置, 左边小于等于 4, 右边大于等于 4

```
l = 0
r = len(a) - 1
pivot = a[l]
while l < r:
    while l < r and a[r] >= pivot:
        r -= 1
    a[l] = a[r]
    while l < r and a[l] <= pivot:
        l += 1
    a[r] = a[l]
a[l] = pivot
```

读原地快速排序时, 重点看:

1. 基准 pivot 是哪个元素。

2. 右指针找小于基准的元素，左指针找大于基准的元素。

3. 最后 pivot 放回 l 的位置。

第一轮分区结束，只能保证基准左边的元素不大于基准、右边的元素不小于基准；左右两段内部不一定有序，还需要递归处理。

九、排序综合比较

设 n 为数据个数， k 为数据值域或桶的数量。

排序	平均时间	最坏时间	稳定性	关键变量与常见考法
选择排序	$O(n^2)$	$O(n^2)$	不稳定	min_i / max_i、每轮结果
冒泡排序	$O(n^2)$	$O(n^2)$	相等不交换时 稳定	i、j、flag、逆序对、循环范围
插入排序	$O(n^2)$	$O(n^2)$	稳定	t、j、元素后移次数
链表插入 排序	$O(n^2)$	$O(n^2)$	可保持稳定	p、q、next 指针填空
计数排序	$O(n+k)$	$O(n+k)$	取决于还原方法	cnt、值域、频次统计
桶排序	与分布和桶内排序有关	与桶内排序有关	取决于桶内处理	bucket[k]、分桶规则
归并排序	$O(n \log n)$	$O(n \log n)$	稳定	i、j、mid、合并两个有序段
快速排序	$O(n \log n)$	$O(n^2)$	不稳定	pivot、l、r、第一轮分区

补充判断：

- 已经基本有序时，优化冒泡排序和插入排序可能接近 $O(n)$ 。
- 选择排序无论原数组是否有序，都要扫描未排序区寻找最小值。
- 归并排序需要额外数组保存合并结果；原地快速排序主要依靠递归调用栈。

十、课堂练习

练习 1：冒泡范围

```
a = [6, 2, 5, 1, 4]
n = len(a)
for i in range(n - 1):
    for j in range(____①____):
        if a[j] > a[j + 1]:
            a[j], a[j + 1] = a[j + 1], a[j]
```

① 应填写：_____。

练习 2: 数组插入排序移动次数

```
a = [1, 3, 5, 2, 4]
t = a[3]
j = 2
cnt = 0
while j >= 0 and a[j] > t:
    a[j + 1] = a[j]
    cnt += 1
    j -= 1
a[j + 1] = t
print(a, cnt)
```

输出结果: _____。